

Proyecto RAG: Asistente de Trámites para Familias con Miembros con Autismo en Andalucía

CRISTÓBAL NAVAS MESA

17 de noviembre de 2025

Memoria del Proyecto

Resumen

Este documento presenta el desarrollo de un sistema Retrieval-Augmented Generation (RAG) como asistente para familias con miembros con Trastorno del Espectro Autista (TEA) en Andalucía. El sistema facilita la consulta de normativa administrativa, ayudas y trámites mediante técnicas de procesamiento de lenguaje natural e inteligencia artificial.

Índice

1. Introducción	3
1.1. Descripción general	3
1.2. Objetivos	3
2. Alcance del Proyecto	3
3. Metodología y Desarrollo	4
3.1. Fases del proyecto	4
3.2. Arquitectura del sistema	4
3.2.1. Componentes principales	4
3.2.2. Flujo de datos	6
3.2.3. Ventajas arquitectónicas	6
4. Herramientas y tecnologías	7
4.1. Lenguajes y frameworks principales	7
4.2. Procesamiento de lenguaje natural	8
4.3. Almacenamiento y bases de datos	8
4.4. Extracción y procesamiento de documentos	8
4.5. Entorno de desarrollo y despliegue	8
4.6. Librerías auxiliares	9
5. Resultados	9
5.1. Funcionamiento del pipeline	9
5.2. Interfaz de usuario	9
5.3. Ejemplos de preguntas y respuestas	9
6. Evaluación y pruebas	9
7. Limitaciones y riesgos	9
8. Impacto social	10
9. Conclusiones y trabajo futuro	10
9.1. Conclusiones	10
9.2. Trabajo futuro	10
10. Consideraciones éticas y legales	10
11. Referencias	10
Anexos	12

1. Introducción

1.1. Descripción general

Las familias con hijos o hijas diagnosticados con Trastorno del Espectro Autista (TEA) en Andalucía enfrentan numerosos desafíos para acceder y comprender los trámites y recursos disponibles. La dispersión de la información en portales web múltiples, documentos en formato PDF, BOJA y guías oficiales dificulta el acceso sencillo y rápido a información actualizada y relevante. Este proyecto nace de la necesidad de crear una herramienta tecnológica que facilite la navegación, búsqueda y comprensión de conceptos normativos y procedimentales, proporcionando un acceso centralizado y amigable mediante técnicas avanzadas de procesamiento del lenguaje natural e inteligencia artificial, concretamente a través de un sistema Retrieval-Augmented Generation (RAG).

1.2. Objetivos

- Recopilar y organizar documentación normativa y de procedimiento relevante para familias con miembros con TEA en Andalucía.
- Procesar documentos en formatos diversos para convertirlos en texto plano limpio.
- Dividir la información en fragmentos o *chunks* para su posterior vectorización.
- Crear una base de datos semántica con embeddings para búsqueda de información.
- Implementar un sistema de búsqueda y generación de respuestas mediante modelos de lenguaje.
- Desarrollar una interfaz sencilla de uso para los usuarios finales.
- Realizar pruebas y validación de la calidad y utilidad del sistema.

2. Alcance del Proyecto

El sistema se centra en los trámites y recursos para familias con miembros con TEA en Andalucía, incluyendo pero no limitado a:

- Reconocimiento de grado de discapacidad.
- Acceso a atención temprana (0-6 años).
- Trámites relacionados con escolarización y NEAE/NNEE.
- Ayudas económicas, prestaciones y apoyos a la autonomía.
- Recursos específicos de asociaciones y federaciones de autismo.

3. Metodología y Desarrollo

3.1. Fases del proyecto

1. **Búsqueda y recopilación de normativa y procedimientos:** Se localizaron fuentes oficiales de información como Junta de Andalucía, BOJA, portales educativos y sociales. Se seleccionaron leyes, decretos, guías, documentos explicativos, Preguntas Frecuentes (FAQs), descargándolos y organizándolos en estructura clara.
2. **Conversión y limpieza de información:** Para asegurar la calidad y uniformidad de los datos, se aplicaron técnicas automatizadas y manuales de extracción y preprocesamiento. Se usaron librerías Python especializadas para extraer texto de PDFs heterogéneos, normalizando codificaciones y eliminando metadatos innecesarios. Se implementaron reglas heurísticas para detectar y eliminar contenido repetitivo, encabezados, pie de página y numeración, garantizando textos limpios y coherentes para las fases posteriores.
3. **Definición de estrategia de chunking:** Segmentación de los textos en fragmentos adecuados para indexar. Se usó un tamaño aproximado de 500 caracteres con solapamiento de 100 caracteres para evitar corte de ideas importantes. Se asociaron metadatos básicos con cada chunk tales como fuente, título, sección y URL.
4. **Creación de embeddings y almacenamiento:** La selección del modelo de embeddings se fundamentó en su eficacia para captar la semántica en español. Se optó por modelos basados en Sentence Transformers adaptados al dominio normativo-social. Los embeddings generados se almacenaron en ChromaDB, una base de datos vectorial de alto rendimiento que facilita búsquedas semánticas rápidas y relevantes mediante técnicas de similitud vectorial.
5. **Implementación del sistema RAG:** Se desarrolló la capa que combina la búsqueda semántica en el vector store y la generación de respuestas naturales con un modelo de lenguaje de gran tamaño (LLM local).
6. **Desarrollo de la interfaz:** Creación de una aplicación web con Flask que proporciona una experiencia usuario amigable, permitiendo consultas en lenguaje natural y visualización enriquecida con fuentes documentales.
7. **Pruebas y evaluación:** Elaboración de preguntas de prueba, así como análisis cualitativo y cuantitativo de la pertinencia y claridad de las respuestas generadas.

3.2. Arquitectura del sistema

El sistema implementa una arquitectura modular basada en el patrón RAG (Retrieval-Augmented Generation), separando claramente las responsabilidades entre componentes independientes pero interconectados. Esta arquitectura facilita el mantenimiento, escalabilidad y modificación de componentes específicos sin afectar al resto del sistema.

3.2.1. Componentes principales

Pipeline de procesamiento de datos Módulo offline responsable de la ingesta, transformación y preparación de documentos normativos:

- **Extractor de documentos:** Descarga automática desde URLs oficiales y conversión de PDF/HTML a texto plano.
- **Limpiador de texto:** Normalización de codificaciones, eliminación de elementos no informativos (encabezados, pies de página, numeración).
- **Chunker:** Segmentación inteligente del texto en fragmentos de 500 caracteres con solapamiento de 100 caracteres para preservar contexto.
- **Generador de embeddings:** Creación de representaciones vectoriales mediante modelos Sentence Transformers optimizados para español.
- **Gestor de metadatos:** Asociación de información contextual (fuente, título, fecha, categoría) a cada chunk procesado.

Base de datos vectorial ChromaDB actúa como almacén persistente y motor de búsqueda semántica:

- Almacenamiento eficiente de embeddings de 384 dimensiones.
- Índice optimizado para búsqueda por similitud coseno.
- Persistencia local sin dependencias externas.
- Recuperación rápida de los k documentos más relevantes (típicamente k=5).

Backend LLM Servicio de generación de respuestas mediante LM Studio:

- Servidor local con API REST compatible con OpenAI.
- Carga de modelos en formato GGUF con cuantización para optimización de memoria.
- Generación de respuestas contextualizadas basadas en fragmentos recuperados.
- Control de temperatura, tokens máximos y otros hiperparámetros.

Capa de aplicación Servidor Flask que orquesta la lógica de negocio:

- Recepción y validación de consultas del usuario.
- Generación de embeddings para la query en tiempo real.
- Coordinación de búsqueda semántica en ChromaDB.
- Construcción de prompts enriquecidos con contexto recuperado.
- Envío de peticiones al LLM y postprocesamiento de respuestas.
- Gestión de historial de conversación y sesiones de usuario.

Interfaz web Frontend responsivo desarrollado con HTML, CSS (Tailwind) y JavaScript vanilla:

- Formulario de consulta en lenguaje natural.
- Visualización de respuestas con formato Markdown.
- Presentación de fuentes documentales citadas.
- Historial de interacciones dentro de la sesión.
- Diseño oscuro optimizado para lectura prolongada.
- Adaptación responsive para dispositivos móviles.

3.2.2. Flujo de datos

El funcionamiento del sistema sigue un flujo claramente definido:

1. Fase de preparación (offline):

- Recopilación de documentos normativos desde fuentes oficiales.
- Procesamiento mediante el pipeline (extracción, limpieza, chunking).
- Generación de embeddings y almacenamiento en ChromaDB.

2. Fase de consulta (online):

- Usuario introduce pregunta en la interfaz web.
- Backend Flask recibe la consulta y genera su embedding.
- Búsqueda semántica en ChromaDB recupera chunks relevantes.
- Construcción de prompt contextualizado con fragmentos recuperados.
- Envío al LLM local que genera respuesta fundamentada.
- Postprocesamiento y presentación al usuario con fuentes citadas.

3.2.3. Ventajas arquitectónicas

Esta arquitectura proporciona varios beneficios clave:

- **Privacidad:** Ejecución completamente local sin envío de datos sensibles a servicios externos.
- **Modularidad:** Cada componente puede actualizarse independientemente.
- **Escalabilidad:** La base vectorial puede ampliarse con nuevos documentos sin modificar el código.
- **Trazabilidad:** Cada respuesta incluye referencias a los documentos fuente originales.
- **Flexibilidad:** Posibilidad de cambiar el modelo LLM sin afectar otros componentes.

Diagrama de arquitectura (ejemplo conceptual)

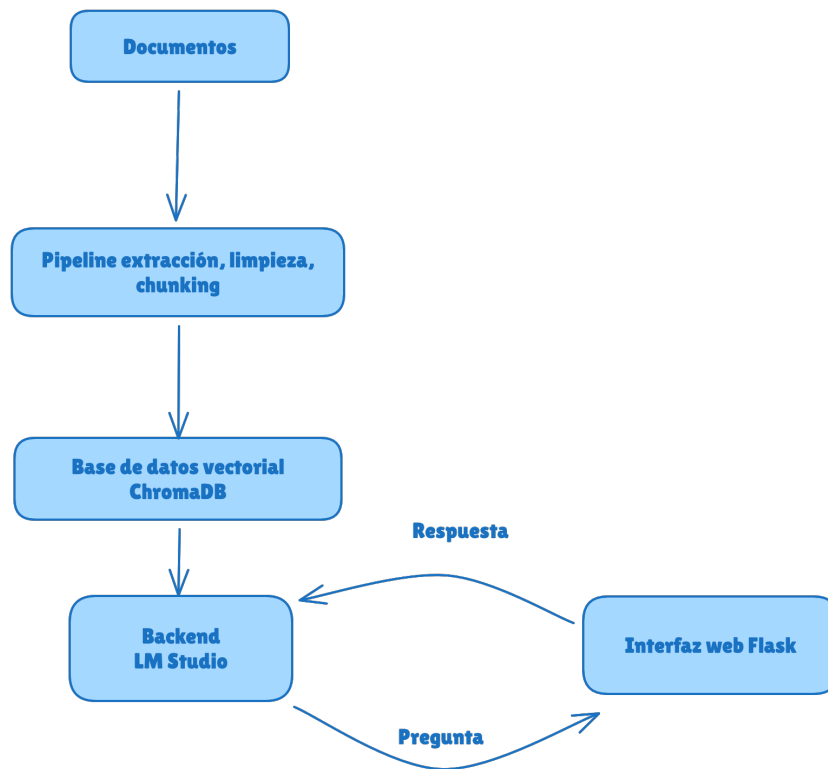


Figura 1: Arquitectura del sistema RAG desarrollado

4. Herramientas y tecnologías

El desarrollo del sistema se ha fundamentado en tecnologías modernas y robustas del ecosistema Python, priorizando soluciones open source y bibliotecas consolidadas en el ámbito del procesamiento de lenguaje natural y desarrollo web.

4.1. Lenguajes y frameworks principales

- **Python 3.10+**: Lenguaje base para todo el desarrollo del pipeline de procesamiento, backend y orquestación de componentes.
- **Flask**: Framework web ligero utilizado para crear la API REST y servir la interfaz de usuario.
- **HTML5, CSS3 y JavaScript**: Tecnologías frontend para la construcción de la interfaz web interactiva.
- **Tailwind CSS**: Framework CSS utility-first para diseño responsivo, accesible y moderno de la interfaz.

4.2. Procesamiento de lenguaje natural

- **LangChain:** Framework para desarrollo de aplicaciones basadas en LLM, facilitando la integración entre modelos, prompts y bases vectoriales.
- **Sentence Transformers:** Biblioteca para generar embeddings semánticos de alta calidad en español mediante modelos como `paraphrase-multilingual-MiniLM-L12-v2`.
- **LM Studio:** Plataforma para ejecución local de modelos de lenguaje grandes (LLM), permitiendo privacidad y control sobre los datos sensibles.
- **Modelos utilizados:** Llama 2, Mistral o modelos equivalentes optimizados para español en formato GGUF para eficiencia.

4.3. Almacenamiento y bases de datos

- **ChromaDB:** Base de datos vectorial embebida para almacenamiento eficiente de embeddings y búsqueda por similitud semántica.
- **SQLite** (opcional): Base de datos relacional para metadatos, logs y configuración del sistema.

4.4. Extracción y procesamiento de documentos

- **PyPDF2 / pdfplumber:** Librerías para extracción de texto de documentos PDF con diferentes estructuras y formatos.
- **BeautifulSoup4:** Parser HTML/XML para extracción de contenido de páginas web oficiales.
- **Requests:** Biblioteca HTTP para descarga automatizada de documentos desde URLs oficiales.
- **python-docx:** Procesamiento de documentos Microsoft Word cuando es necesario.
- **Regex (re):** Expresiones regulares para limpieza, normalización y estructuración de texto extraído.

4.5. Entorno de desarrollo y despliegue

- **Git:** Control de versiones para gestión del código fuente.
- **Visual Studio Code:** IDE principal con extensiones para Python y desarrollo web.
- **Jupyter Notebooks:** Entorno interactivo para experimentación, análisis y visualización de datos durante el desarrollo.
- **Virtual Environment (venv):** Gestión de dependencias y aislamiento del entorno Python.

4.6. Librerías auxiliares

- **NumPy**: Operaciones numéricas y manejo eficiente de arrays para procesamiento de embeddings.
- **Pandas**: Manipulación y análisis de datos tabulares para organización de metadatos.
- **python-dotenv**: Gestión segura de variables de entorno y configuraciones sensibles.
- **Logging**: Módulo estándar de Python para registro de eventos, errores y seguimiento del sistema.

5. Resultados

5.1. Funcionamiento del pipeline

El pipeline ha sido capaz de procesar una amplia colección de documentos normativos sobre TEA en Andalucía, limpiando, segmentando y generando embeddings eficientes para búsqueda semántica. Esto ha permitido construir una base documental accesible para sistemas de generación de respuestas.

5.2. Interfaz de usuario

La interfaz web permite consultas en lenguaje natural, mantiene un historial de preguntas y respuestas, y muestra referencias legibles a fuentes documentales. Su diseño oscuro y responsivo mejora la experiencia de usuario y accesibilidad.

5.3. Ejemplos de preguntas y respuestas

Se validaron preguntas relacionadas con trámites destacando:

- Procedimientos para solicitar atención temprana.
- Ayudas económicas disponibles para personas con TEA.
- Pasos para el reconocimiento de discapacidad.

Las respuestas generadas incluyen fragmentos normativos precisos que facilitan la comprensión y la confianza del usuario.

6. Evaluación y pruebas

Se definió un conjunto de preguntas típicas para familias con TEA y se analizaron respuestas en términos de relevancia, precisión y claridad. Se llevó a cabo una evaluación cualitativa con expertos y usuarios.

7. Limitaciones y riesgos

El sistema depende de la calidad de la base documental y requiere actualización constante. Existen riesgos relacionados con la interpretación incorrecta por parte del usuario final y limitaciones del modelo de lenguaje.

8. Impacto social

Este sistema apoya la inclusión, autonomía e información de familias TEA, facilitando el acceso a recursos públicos y legales de forma comprensible.

9. Conclusiones y trabajo futuro

9.1. Conclusiones

El proyecto ha demostrado la viabilidad de usar técnicas RAG en el ámbito social para mejorar el acceso a la información a colectivos vulnerables.

9.2. Trabajo futuro

Se plantea incorporar streaming en las respuestas, APIs REST para integración multiplataforma, ampliación documental y procesamiento multilingüe.

10. Consideraciones éticas y legales

El sistema es un apoyo informativo y no reemplaza asesoramiento profesional. Es esencial mantener un lenguaje claro, respetuoso y la actualización normativa.

11. Referencias

- Ley 1-2023 Atención Temprana
- Ley 4-2017 Derechos Discapacidad
- Decreto 147-2002 Atención Educativa NEAE
- Solicitud Grado Discapacidad
- Derechos y Prestaciones Discapacidad
- Centros de Valoración CVO
- Guía Prestaciones TEA Mírame PDF
- Guía Cómo Solicitar Discapacidad
- Solicitud Dependencia
- Ventanilla Electrónica Dependencia VED
- Guía Ley Dependencia UNIR
- Texto Ley 1-2023 Atención Temprana BOJA
- Portal Familias Necesidades Especiales
- Listado Centros CAIT

- Instrucción 2015 Detección Educativa
- Portal Escuela Familias TEA
- Portal Familias Consejería Educación
- Convenio Educación Autismo Andalucía 2020
- Comunicado Apoyo Alumnado TEA Autismo Andalucía
- Becas NEAE Ministerio
- Ayudas Prestaciones Discapacidad JA
- Becas NEAE Andalucía Educaorienta
- Ayudas Contratación Cuidadores
- Prestaciones Estatales Discapacidad SS
- Autismo Andalucía Servicios
- Autismo Sevilla Apoyos
- Autismo Sevilla Apoyo Familiares

Anexos

Anexo A: Pipeline de Procesamiento de Documentos

Este anexo presenta el código del módulo de extracción y procesamiento de documentos normativos. El sistema es capaz de procesar tanto archivos PDF directos como páginas HTML que contienen enlaces a PDFs anidados.

A.1 Script Principal de Extracción

```
1 import os
2 import json
3 import requests
4 import fitz # PyMuPDF
5 import trafilatura
6 import re
7 import logging
8 from bs4 import BeautifulSoup
9 from urllib.parse import urljoin
10
11 from config import (
12     TEXTOS_LIMPIOS_DIR as OUTPUT_DIR,
13     DOCS_ORIGINALES_DIR as OUTPUT_ORIGIN_PDFS,
14     REQUEST_HEADERS,
15     SOURCES_LIST
16 )
17
18 # Cargar lista de fuentes desde archivo JSON
19 with open(SOURCES_LIST, "r", encoding="utf-8") as f:
20     SOURCES = json.load(f)
21
22 def clean_text(text: str) -> str:
23     """Limpia el texto de espacios en blanco anómalos."""
24     if not text:
25         return ""
26     text = re.sub(r'\n\s*\n', '\n\n', text)
27     text = re.sub(r'[ \t]+', ' ', text)
28     text = '\n'.join([line.strip() for line in text.split('\n')])
29     text = re.sub(r'\n\n+', '\n\n', text).strip()
30     return text
31
32 def extract_text_from_pdf(url: str) -> str:
33     """Extrae y limpia el texto de una URL PDF."""
34     try:
35         response = requests.get(url, headers=REQUEST_HEADERS, timeout
36 =15)
37         response.raise_for_status()
38         pdf_bytes = response.content
39         full_text = ""
40
41         with fitz.open(stream=pdf_bytes, filetype="pdf") as doc:
42             logging.info(f"Procesando PDF {url} ({doc.page_count}
43 paginas)...")
44             for page_num, page in enumerate(doc):
45                 page_text = page.get_text("text")
46                 lines = page_text.split('\n')
```

```

46         # Filtrar lineas muy cortas o solo numeros
47         cleaned_lines = [
48             line for line in lines
49             if len(line.strip()) >= 5 or not any(char.isdigit()
for char in line)
50         ]
51         page_text_cleaned = '\n'.join(cleaned_lines)
52         full_text += page_text_cleaned + f"\n\n--- (Fin Pagina {
page_num + 1}) ---\n\n"
53
54         return clean_text(full_text)
55     except requests.exceptions.RequestException as e:
56         logging.error(f"Error al descargar PDF {url}: {e}")
57         return ""
58     except Exception as e:
59         logging.error(f"Error procesando PDF {url}: {e}")
60         return ""
61
62 def download_pdf_original(url: str, title: str):
63     """Descarga y guarda el PDF original si la URL es .pdf."""
64     try:
65         if not os.path.exists(OUTPUT_ORIGIN_PDFS):
66             os.makedirs(OUTPUT_ORIGIN_PDFS)
67
68         response = requests.get(url, headers=REQUEST_HEADERS, stream=
True, timeout=15)
69         response.raise_for_status()
70
71         safe_filename = re.sub(r'[^a-z0-9]+', '_', title.lower().strip()
) + ".pdf"
72         filepath = os.path.join(OUTPUT_ORIGIN_PDFS, safe_filename)
73
74         with open(filepath, "wb") as f:
75             for chunk in response.iter_content(chunk_size=2048):
76                 if chunk:
77                     f.write(chunk)
78             logging.info(f"PDF original guardado en: {filepath}")
79     except Exception as e:
80         logging.error(f"Error al descargar PDF original {url}: {e}")
81
82 def process_html_and_nested_pdfs(url: str, title: str) -> str:
83     """
84     Extrae el texto principal de una pagina HTML y PDFs anidados.
85     Tambien descarga los PDFs anidados.
86     """
87     try:
88         response = requests.get(url, headers=REQUEST_HEADERS, timeout
=15)
89         response.raise_for_status()
90         html_content = response.text
91
92         logging.info(f"Extrayendo texto principal de HTML: {url}")
93         main_page_text = trafilatura.extract(
94             html_content,
95             include_comments=False,
96             include_tables=True,
97             deduplicate=True
98         ) or ""

```

```

99
100     # Buscar PDFs anidados
101     soup = BeautifulSoup(html_content, 'html.parser')
102     pdf_links = []
103     for a_tag in soup.find_all('a', href=True):
104         href = a_tag['href']
105         if href.lower().endswith('.pdf'):
106             absolute_pdf_url = urljoin(url, href)
107             pdf_links.append(absolute_pdf_url)
108
109     nested_pdf_text = ""
110     if pdf_links:
111         unique_links = set(pdf_links)
112         logging.info(f"Se encontraron {len(unique_links)} PDFs
anidados unicos.")
113
114         for pdf_url in unique_links:
115             short_title = f"{title}_anidado"
116             download_pdf_original(pdf_url, short_title)
117             logging.info(f"Procesando PDF anidado: {pdf_url}")
118
119             nested_pdf_text += f"\n\n--- INICIO PDF ANIDADO ({
pdf_url}) ---\n\n"
120             nested_pdf_text += extract_text_from_pdf(pdf_url)
121             nested_pdf_text += f"\n\n--- FIN PDF ANIDADO ({pdf_url})
---\n\n"
122         else:
123             logging.info("No se encontraron PDFs anidados.")
124
125     combined_text = main_page_text + nested_pdf_text
126     return clean_text(combined_text)
127 except requests.exceptions.RequestException as e:
128     logging.error(f"Error al descargar HTML {url}: {e}")
129     return ""
130 except Exception as e:
131     logging.error(f"Error procesando HTML {url}: {e}")
132     return ""
133
134 def save_text_to_file(title: str, content: str, url: str):
135     """Guarda el contenido limpio en un archivo .txt."""
136     if not content:
137         logging.warning(f"No se guardo contenido para '{title}' (URL: {
url}) porque estaba vacio.")
138         return
139
140     if not os.path.exists(OUTPUT_DIR):
141         os.makedirs(OUTPUT_DIR)
142
143     safe_filename = re.sub(r'[a-z0-9]+', '_', title.lower().strip()) +
".txt"
144     filepath = os.path.join(OUTPUT_DIR, safe_filename)
145
146     try:
147         with open(filepath, 'w', encoding='utf-8') as f:
148             f.write(f"FUENTE_ORIGINAL: {url}\n")
149             f.write(f"TITULO: {title}\n")
150             f.write("-----\n\n")
151             f.write(content)

```

```

152     logging.info(f"Texto limpio guardado en: {filepath}")
153 except IOError as e:
154     logging.error(f"Error al guardar archivo {filepath}: {e}")
155
156 def main():
157     if not os.path.exists(OUTPUT_DIR):
158         os.makedirs(OUTPUT_DIR)
159     if not os.path.exists(OUTPUT_ORIGIN_PDFS):
160         os.makedirs(OUTPUT_ORIGIN_PDFS)
161
162     logging.info(f"Iniciando procesamiento de {len(SOURCES)} fuentes..."
163 )
164
165     for source in SOURCES:
166         title = source['title']
167         url = source['url']
168         logging.info(f"Procesando: '{title}' ({url}")
169
170         cleaned_content = ""
171         if url.lower().endswith('.pdf'):
172             cleaned_content = extract_text_from_pdf(url)
173             download_pdf_original(url, title)
174         else:
175             cleaned_content = process_html_and_nested_pdfs(url, title)
176
177         save_text_to_file(title, cleaned_content, url)
178
179     logging.info("--- Proceso completado ---")
180
181 if __name__ == "__main__":
182     main()

```

Listing 1: Módulo de extracción y limpieza de documentos (extract_and_clean.py)

Descripción del módulo

Este módulo implementa el pipeline de extracción y limpieza de documentos con las siguientes funcionalidades clave:

- **Extracción de PDFs:** Utiliza PyMuPDF (fitz) para extraer texto de documentos PDF descargados desde URLs oficiales.
- **Procesamiento HTML:** Emplea Trafilatura para extraer contenido principal de páginas web, eliminando elementos de navegación y publicidad.
- **PDFs anidados:** Detecta automáticamente enlaces a PDFs dentro de páginas HTML y los procesa recursivamente.
- **Limpieza de texto:** Normaliza espacios en blanco, elimina líneas muy cortas o numéricas que suelen ser artefactos de maquetación.
- **Almacenamiento organizado:** Guarda tanto los PDFs originales como el texto limpio en directorios separados con metadatos de fuente.

Anexo B: Generación de Embeddings y Almacenamiento Vectorial

Este módulo implementa la generación de representaciones vectoriales (embeddings) de los fragmentos de texto procesados y su almacenamiento en ChromaDB para búsqueda semántica eficiente.

B.1 Script de Generación y Almacenamiento de Embeddings

```
1 import json
2 import os
3 from sentence_transformers import SentenceTransformer
4 import chromadb
5 from chromadb.config import Settings
6 from config import CHUNKS_JSON, CHUNKS_OUTPUT
7
8 def load_chunks(json_path):
9     """Carga los chunks de texto desde el archivo JSON."""
10     with open(json_path, 'r', encoding='utf-8') as f:
11         chunks = json.load(f)
12     return chunks
13
14 def embed_texts(texts, model):
15     """
16     Genera embeddings para una lista de textos usando el modelo
17     especificado.
18     Muestra barra de progreso durante el proceso.
19     """
20     return model.encode(texts, show_progress_bar=True)
21
22 def main():
23     # Verificar existencia del archivo de chunks
24     if not os.path.exists(CHUNKS_JSON):
25         print(f"No se encontro el archivo de chunks: {CHUNKS_JSON}")
26         return
27
28     # Cargar chunks procesados
29     chunks = load_chunks(CHUNKS_JSON)
30     texts = [chunk["text"] for chunk in chunks]
31     ids = [f"{chunk['source']}_{chunk['chunk_id']}" for chunk in chunks]
32     metadatas = [
33         {"source": chunk["source"], "chunk_id": chunk["chunk_id"]}
34         for chunk in chunks
35     ]
36
37     # Cargar modelo de embeddings
38     print(f"Cargando modelo de embeddings...")
39     model = SentenceTransformer('all-MiniLM-L6-v2')
40
41     # Generar embeddings para todos los chunks
42     print(f"Generando embeddings para {len(texts)} chunks...")
43     embeddings = embed_texts(texts, model)
44
45     # Inicializar cliente ChromaDB con persistencia local
46     client = chromadb.PersistentClient(
47         path=CHUNKS_OUTPUT,
48         settings=Settings()
```

```

49
50 # Obtener o crear coleccion para documentos
51 collection = client.get_or_create_collection("normativa_andalucia")
52
53 # Insertar embeddings en la coleccion
54 print("Insertando embeddings en la coleccion...")
55 collection.add(
56     documents=texts,
57     embeddings=embeddings.tolist(),
58     metadatas=metadatas,
59     ids=ids
60 )
61
62 print(f"Embeddings guardados persistentemente en {CHUNKS_OUTPUT}")
63
64 if __name__ == "__main__":
65     main()

```

Listing 2: Módulo de embeddings y ChromaDB (generate_embeddings.py)

Descripción del módulo

Este módulo implementa la capa de representación vectorial y almacenamiento con las siguientes características:

- **Carga de chunks:** Lee los fragmentos de texto previamente generados desde archivo JSON estructurado.
- **Modelo de embeddings:** Utiliza all-MiniLM-L6-v2, un modelo ligero de Sentence Transformers que genera vectores de 384 dimensiones, equilibrando calidad y eficiencia computacional.
- **Generación de embeddings:** Procesa todos los chunks de texto en batch, mostrando progreso durante la operación para grandes volúmenes de datos.
- **Almacenamiento persistente:** Configura ChromaDB como base de datos vectorial local con:
 - Persistencia en disco para mantener datos entre ejecuciones
 - Colección específica para normativa andaluza
 - Asociación de metadatos (fuente, ID) con cada embedding
 - Identificadores únicos compuestos por fuente y chunk ID
- **Escalabilidad:** La arquitectura permite añadir nuevos documentos incrementalmente sin reconstruir toda la base vectorial.

Consideraciones técnicas

El modelo all-MiniLM-L6-v2 fue seleccionado por:

- Tamaño compacto (solo 80 MB) que permite ejecución local eficiente
- Buenos resultados en tareas de similitud semántica en español

- Balance entre calidad de representación y velocidad de inferencia
- Compatibilidad con búsqueda por similitud coseno en ChromaDB

ChromaDB proporciona:

- Almacenamiento eficiente de vectores de alta dimensionalidad
- Búsqueda k-nearest neighbors optimizada
- API simple compatible con operaciones de inserción y consulta
- Sin dependencias de servidores externos